

**LAPORAN PRAKTIKUM UJIAN TENGAH SEMESTER
MATA KULIAH COMMUNICATION PROTOCOL**



Nama Mahasiswa	: Salsabila Tahira Yasmin
NIM	: 25120500016
Kelas	: ComP2
Judul Case	: User/Profile
Link GitHub	: bit.ly/GitHubUTS

**UNIVERSITAS CAKRAWALA
2026**

1. Ringkasan Case

a. Tujuan API

Case yang dipilih adalah user/profile dengan menggunakan simulasi REST API <http://127.0.0.1:8088/api/users>. Di mana di sini akan dilakukan pengujian method GET untuk melihat daftar user baik secara keseluruhan maupun detail, method POST untuk membuat atau menambahkan user baru dan PATCH untuk mengupdate profil user.

b. Skenario Data/Bisnis

- Data diminta oleh klien (dalam praktik ini postman) dengan server <http://127.0.0.1:8088/api/userss>, menggunakan protokol HTTP.
- Ekspektasi kondisi data adalah valid dan menghasilkan respon 200 atau 201, yang berarti respon sukses atau data baru berhasil ditambahkan.
- Pada praktik ini penyusun akan melihat daftar semua user tanpa filter atau penambahan parameter, kemudian detail user, menambahkan user baru dan mengupdate profil user.
- Penyusun juga akan mengamati

2. Desain Request

• GET daftar user

Komponen	Deskripsi
Endpoint	http://127.0.0.1:8088/api/users/
Method	GET
Headers	-
Body	-
Expected Response	Status 200 ok, body response berisi semua data user

• GET detail user

Komponen	Deskripsi
Endpoint	http://127.0.0.1:8088/api/users/2
Method	GET
Headers	-
Body	-
Expected Response	Status 200 ok, body response berisi data user dengan id nomor 2

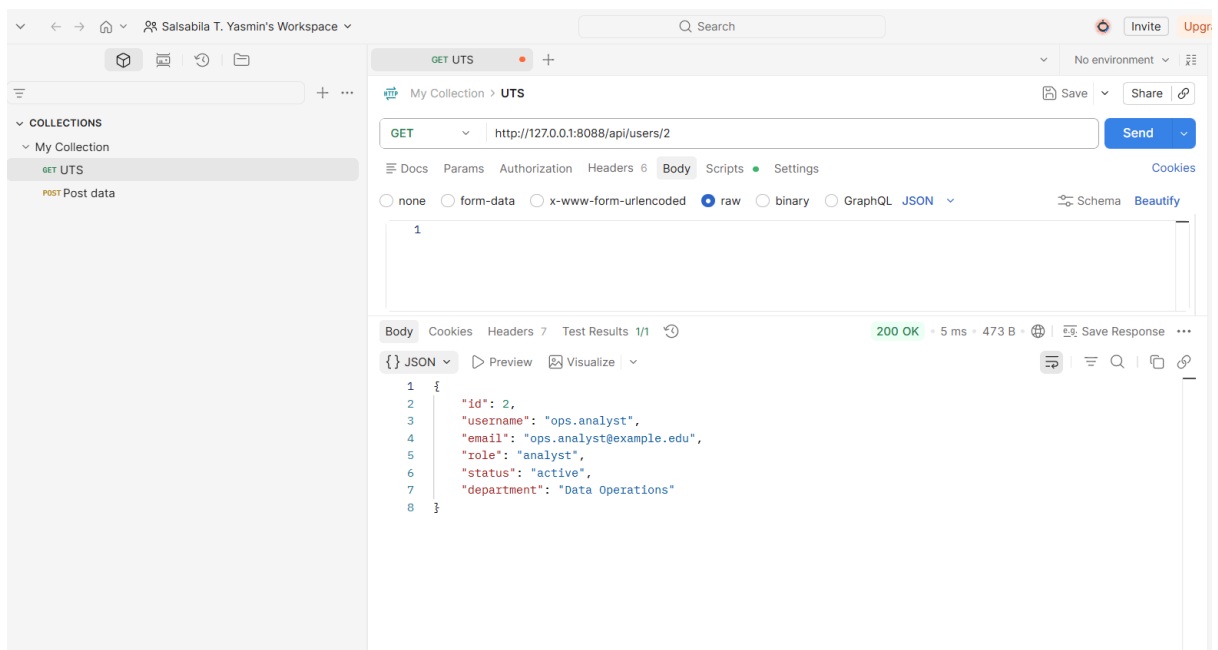
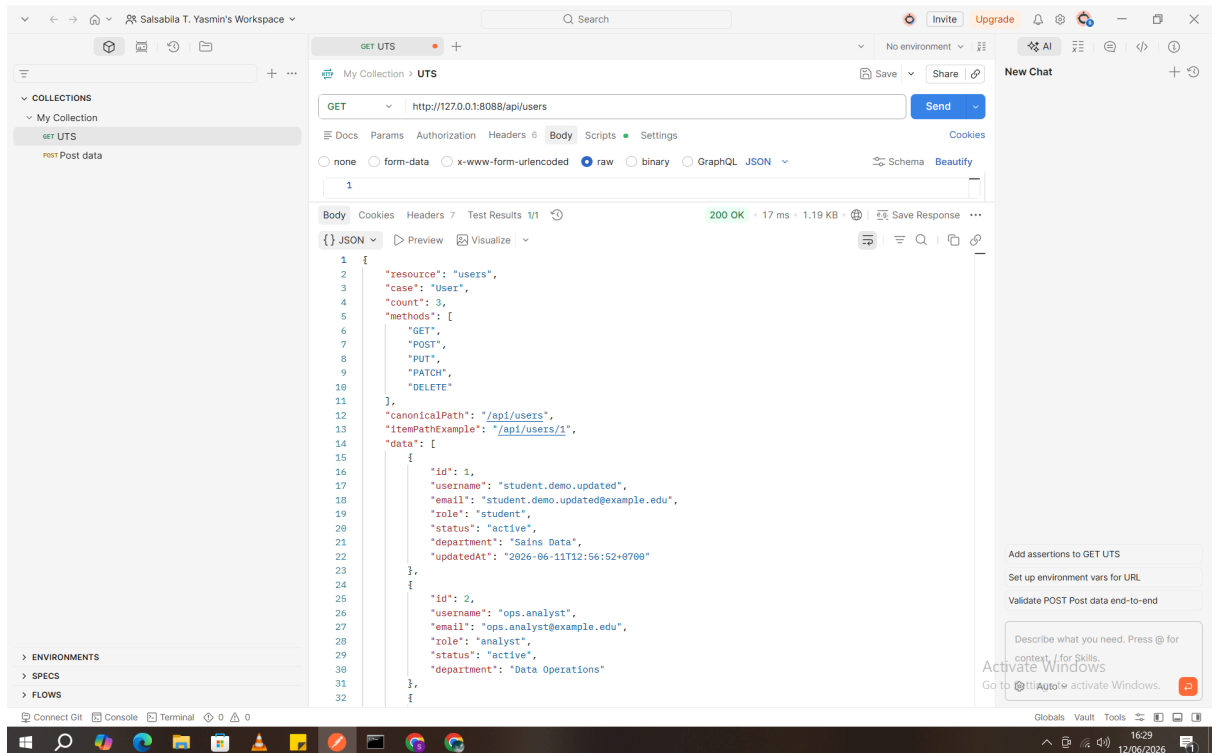
- POST create user

Komponen	Deskripsi
Endpoint	http://127.0.0.1:8088/api/users/
Method	POST
Headers	content-type application/json
Body	{ "username": "data.student", "email": "data.student@example.edu", "role": "student", "status": "active", "department": "Sains Data" }
Expected Response	Status 201 created, body response berisi data user dengan id yang baru

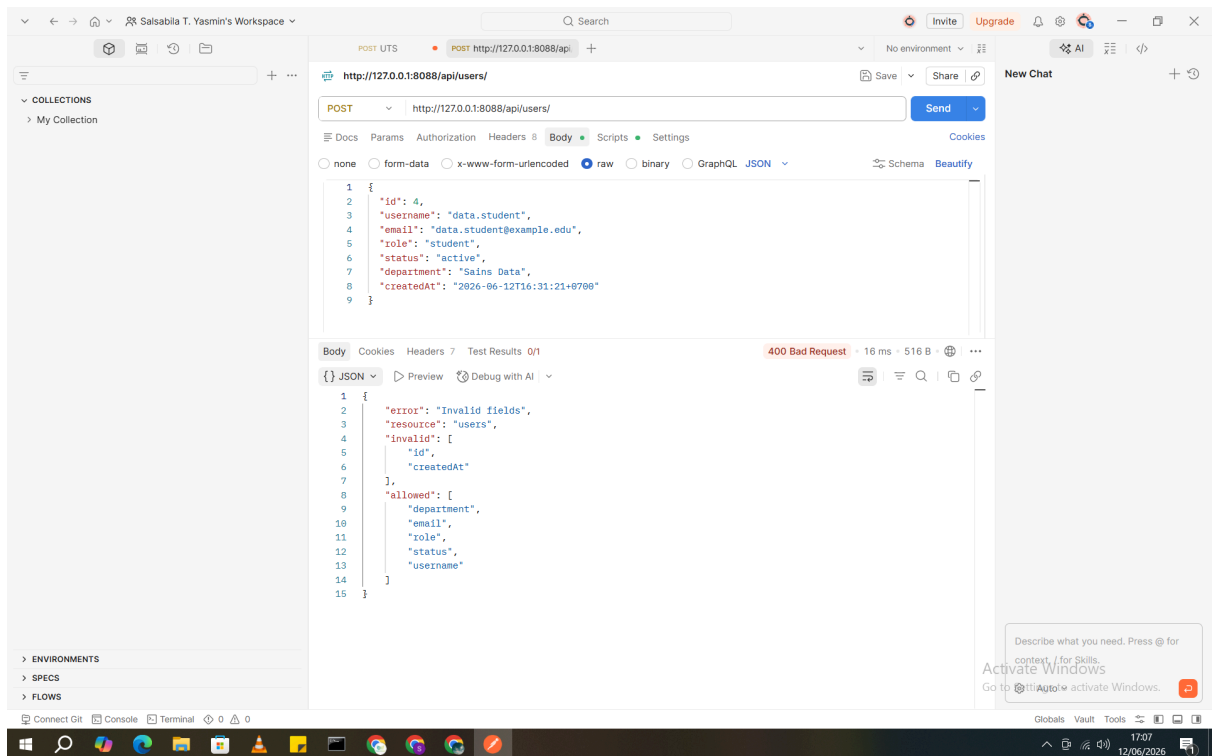
- PATCH update profile

Komponen	Deskripsi
Endpoint	http://127.0.0.1:8088/api/users/1
Method	PATCH
Headers	content-type application/json
Body	{ "status": "inactive" }
Expected Response	Status 200 oke, data dengan id 1 diupdate pada field status

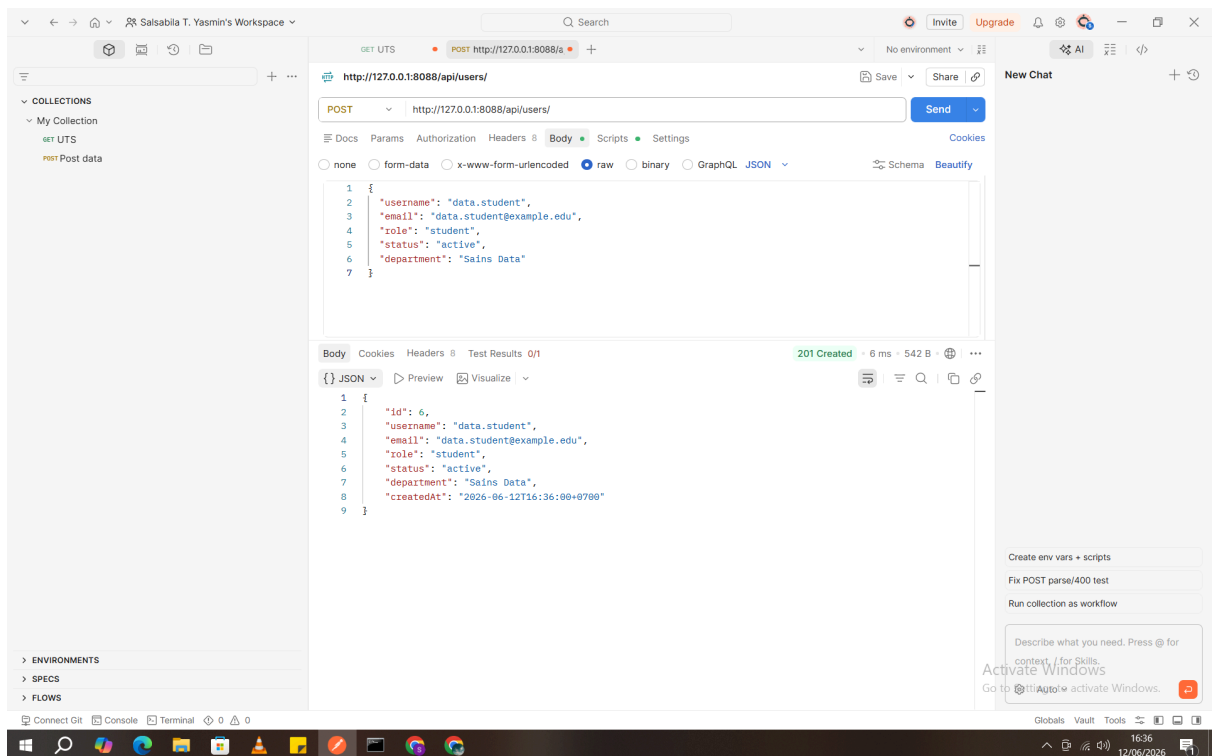
3. Evidence GET



4. Evidence POST

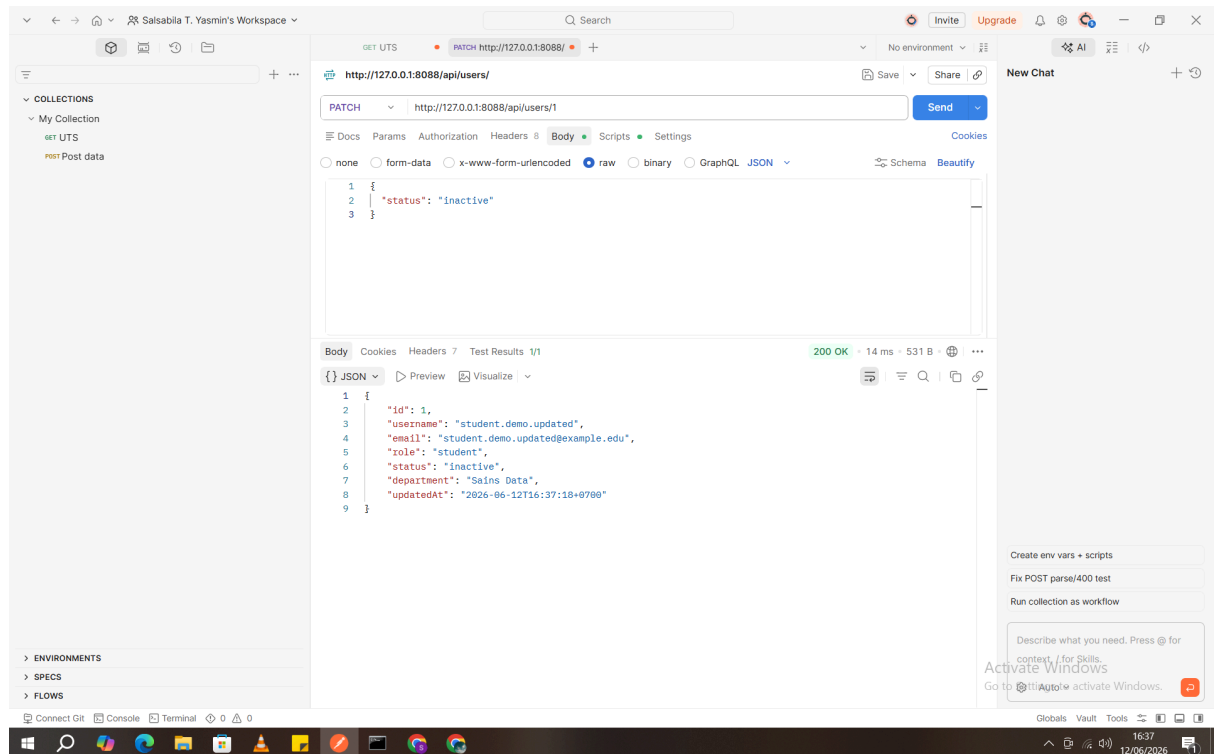


Method POST untuk menambahkan data user baru.1



Method POST untuk menambahkan data user baru.2

5. Evidence PATCH



Method PATCH untuk mengupdate data user 1 pada field status

6. Analisis Respons

Berdasarkan empat request yang telah dijalankan (GET all users, GET user id=2, POST create user, PATCH update user id=1), berikut analisis masing-masing respons.

a. GET daftar semua user

- Status Code: 200 OK
- Headers: Content-Type: application/json menunjukkan bahwa server mengirim data dalam format JSON. Tidak ada header Authorization karena endpoint bersifat publik.
- Body: Berupa array JSON yang berisi objek-objek user. Setiap objek memiliki field: id, username, email, role, status, department, createdAt.
- Makna: Request berhasil, klien menerima seluruh data user yang tersedia di server.

b. GET detail user dengan id = 2

- Status Code: 200 OK
- Headers: Sama seperti di atas, Content-Type: application/json.

- Body: Objek JSON tunggal yang memuat data user dengan id: 2. Jika id tidak ditemukan, server akan mengembalikan 404 Not Found dengan pesan error.
- Makna: Server berhasil mengembalikan data spesifik sesuai parameter id.

c. POST create user (data valid)

- Status Code: 201 Created
- Headers: Content-Type: application/json dan biasanya Location header berisi URL resource baru (misal /api/users/4).
- Body: Server mengembalikan objek user yang baru saja dibuat, lengkap dengan id dan createdAt yang digenerate oleh server. Field id bersifat unik dan otomatis bertambah.
- Makna: Resource baru berhasil diciptakan. Klien wajib mengirim hanya field yang diizinkan (username, email, role, status, department). Jika mengirim id atau createdAt, server akan menolak dengan 400 Bad Request dan menyebutkan field invalid (sesuai pengalaman praktik sebelumnya).

d. PATCH update user

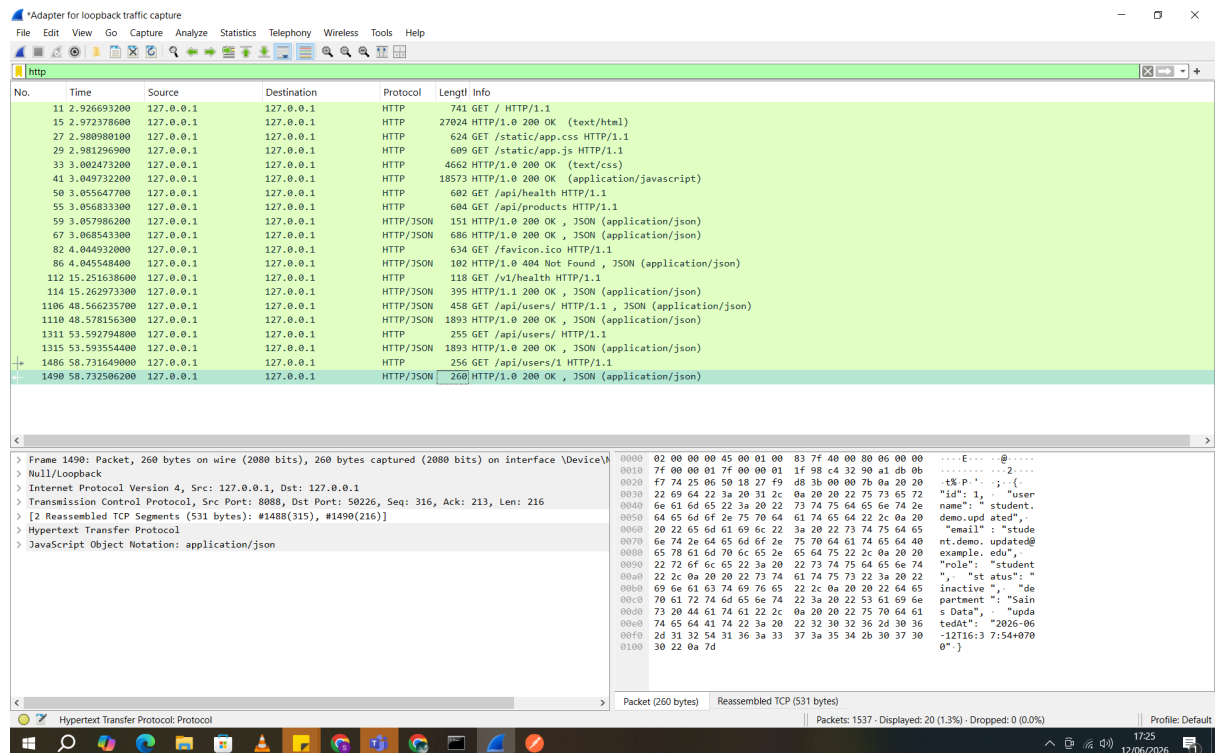
- Status Code: 200 OK
- Headers: Content-Type: application/json.
- Body: Biasanya server mengembalikan objek user yang sudah diperbarui. Pada kasus ini, hanya field status yang berubah menjadi "inactive", sedangkan field lain tetap.
- Makna: Update parsial berhasil. Metode PATCH tepat digunakan karena hanya mengubah satu field. Jika menggunakan PUT, seharusnya mengirim seluruh objek.

e. Penanganan Error

Saat mencoba POST dengan mengirim id dan createdAt, server merespons:

- a. Status Code: 400 Bad Request
- b. Body: {"error": "Invalid fields", "invalid": ["id", "createdAt"], "allowed": [...]}
- c. Analisis: Server memiliki validasi ketat – field yang bersifat sistem (auto-generated) tidak boleh dikirim oleh klien. Ini melatih praktik REST yang baik.

7. Analisis Encoding/Traffic



Pada paket nomor 1490, server mengembalikan status 200 OK dengan tipe konten application/json. Payload JSON yang terlihat di frame hex menunjukkan field id, username, email, role, status, department, dan updatedAt. Ini sesuai dengan response dari endpoint /api/users/1.

8. Kesimpulan

Berdasarkan praktikum User/Profile API menggunakan server lokal <http://127.0.0.1:8088> dan tool Postman/Wireshark, dapat disimpulkan beberapa hal:

1. Pemahaman REST API

- Method GET, POST, PATCH digunakan sesuai dengan semantiknya: GET untuk membaca, POST untuk membuat, PATCH untuk update parsial.
- Status code memberikan informasi yang jelas tentang hasil request (200, 201, 400, 404).

2. Validasi Data oleh Server

- Server menolak field id dan createdAt pada request POST karena kedua field tersebut merupakan tanggung jawab server (auto-generated). Ini adalah desain API yang baik.
- Client hanya perlu mengirim field yang diizinkan, sehingga integritas data terjaga.

3. Wireshark membantu memverifikasi lalu lintas HTTP di level paket, memperkuat bukti praktik.

4. Kendala dan Solusi

- Awalnya terjadi error 400 Bad Request karena mengirim field id dan createdAt. Solusinya dengan membaca pesan error dan menyesuaikan body request.
 - Pada Wireshark, lalu lintas localhost tidak terlihat di antarmuka Wi-Fi/Ethernet. Solusinya menginstal Npcap dengan opsi Support loopback traffic dan memilih antarmuka Adapter for loopback traffic capture.
5. Pembelajaran untuk Proyek ke Depan
- Selalu perhatikan dokumentasi API (field apa saja yang diizinkan).
 - Gunakan environment variables di Postman untuk memudahkan testing.
 - Memanfaatkan traffic capture untuk troubleshooting koneksi atau memeriksa payload.

Kesimpulan akhir: Praktikum ini berhasil mencapai tujuan yaitu menjalankan minimal 2 GET dan 2 POST/PATCH, menganalisis respons, serta menangkap traffic. Semua evidence terdokumentasi dengan baik dalam laporan, screenshot, dan repository GitHub. Dengan pemahaman ini, mahasiswa siap mengimplementasikan komunikasi data pada sistem nyata menggunakan REST API berbasis JSON.